

WHAMI — developer requirements & build guide

Flutter implementation plan, mapped from client requirements and the project architecture doc · prepared for the IRHASOFT dev team

Platform requirement

Requirement: native app on Android + iOS, not a PWA — must work fully offline and survive iOS background/cache eviction.

How: build in Flutter. It compiles to native ARM binaries on both platforms (not a browser shell), giving full native filesystem and sensor access. Use `path_provider` for all persistent storage — never rely on browser-style cache or web storage APIs. One codebase, two store builds: `flutter build appbundle` for Play Store, `flutter build ios` for App Store.

Open source preference: every package below is open source except the AR engines (ARCore/ARKit) underneath the AR plugin — those are free but closed-source, and unavoidable since nothing open source matches their tracking quality.

Core modules

1. RegionPackManager

Requirement: download, verify, update, and audit offline region packs containing maps, seamap, landmark, magnetic, celestial, and route-trust data.

How: structure each pack as the folder layout from the project doc (`/maps` , `/marine` , `/landmarks` , `/magnetic` , `/celestial` , `/imu` , `/trust`), zipped for download. Use `dio` or `http` to download, `archive` to unzip, `path_provider` to store. Use `crypto` to verify signed manifest hashes (the data-moat requirement). Store manifests and version history in `sqflite` or `hive` .

2. SensorHub

Requirement: collect GPS/GNSS, IMU, gyroscope, magnetometer, barometer, and report sensor availability/health.

How: `geolocator` for GPS/GNSS; `sensors_plus` for accelerometer, gyroscope, magnetometer (wraps Core Motion / SensorManager natively). Barometer isn't reliably covered by `sensors_plus` — verify per platform, may need a small native platform channel. Wrap all of this in one Dart `SensorHub` service exposing unified streams plus a health/availability status per sensor.

3. LandmarkMappingModule

Requirement: visual feature extraction and landmark/seamap matching (was CameraX/ARCore/OpenCV in the Android starter).

How: `camera` package for capture, cross-platform. For matching: `tflite_flutter` with an on-device landmark recognition model, or `opencv_dart` for classical feature matching (ORB/SIFT) if no trained model is ready yet — TFLite is simpler to maintain long-term. Store landmark feature descriptors locally for matching against the loaded region pack.

4. MagneticModule

Requirement: magnetic heading, field stability, fingerprinting, and a path to future anomaly-map matching.

How: `sensors_plus` magnetometer stream. Build calibration (figure-8 routine), fingerprinting, and core-field subtraction against the EMAG2 baseline shipped in the region pack — plain Dart logic. Keep this behind a clean `MagneticModule` interface so future external hardware (fluxgate/quantum sensors in WHAMI Pocket/Atlas/Pro) can feed in later over Bluetooth (`flutter_blue_plus`) without rewriting the app.

5. CelestialModule

Requirement: sun/moon/star/sextant-style position opinion with sky stability scoring, always shown with a wide uncertainty circle, never false precision.

How: implement the almanac/solar-position calculation in pure Dart (a small ported open algorithm). Ship almanac tables inside the region pack's `/celestial` folder. Bake the wide-uncertainty-circle behavior into the data model itself, not just a UI styling choice.

6. TrustFusionEngine

Requirement: combine all source opinions, check residuals, de-weight suspicious sources, output the trusted position estimate.

How: pure Dart module, no platform dependency. Start with the weighted formula from the doc: $\text{Trust} = 0.45 \times \text{Landmark} + 0.20 \times \text{GPS} + 0.15 \times \text{Magnetic} + 0.10 \times \text{IMU} + 0.10 \times \text{Sky}$. Make weights dynamic — landmark weight drops at night/at sea, redistributing to magnetic/sky; magnetic trust drops indoors near metal; GPS trust drops sharply on an impossible jump. Plan a v2 milestone for a full Extended Kalman Filter / factor-graph approach — Dart's linear algebra ecosystem is thin (`ml_linalg` is the closest option), worth a short research spike first.

7. UI layer — one-view position opinions

Requirement: single map screen with color-coded markers and uncertainty circles (amber = WHAMI trusted, black = landmark, blue = GPS, red = magnetic, green = sky, purple = IMU), plus an expandable "position opinions" list.

How: `maplibre_gl` for the map with custom marker overlays per source. Render uncertainty circles as MapLibre native circle layers so radius scales correctly in meters when zooming. The opinions list below the map is a simple list of cards, one per source, each showing name, confidence %, and radius.

8. App pages

Requirement: Map, Landmark Scan, AR Scan, Sensors, Region Pack — five screens, per the existing starter's page set.

How: five Flutter routes navigated with `go_router`. Keep each screen thin — it should read from the relevant module, not contain business logic itself.

9. AR scan mode

Requirement: separate AR screen with live camera feed behind a reticle and plane/pose/quality status.

How: `ar_flutter_plugin` (wraps ARCore on Android, ARKit on iOS). Use plane detection and pose tracking only — don't build around LiDAR scene reconstruction. LiDAR is Pro-iPhone-only and nearly absent on Android, while ARCore's Depth API already covers the large majority of active Android devices and ARKit ships free on every iPhone since the 6s, so motion tracking alone gives broad coverage without a LiDAR dependency.

Screen wireframes

Rough low-fidelity sketches of the five app screens, for layout reference only — not final visual design.

Map

WHAMI trusted

82%

GPS / GNSS

78%

Map

Scan

AR

Sensors

Packs

Sensors

GPS

Magnetometer

IMU

Sky / sextant

Map

Scan

AR

Sensors

Packs

Landmark scan

Harbor tower

saved

Depot roofline

saved

Map

Scan

AR

Sensors

Packs

AR scan

Plane found

Pose quality

Good

Map

Scan

AR

Sensors

Packs

Region pack

SF Bay area

1.2GB

Mountain lake

download

Map

Scan

AR

Sensors

Packs

Package reference

Need	Package
Maps / offline tiles	maplibre_gl
Sensors (IMU, magnetometer)	sensors_plus
GPS/GNSS	geolocator
Camera capture	camera
On-device landmark matching	tflite_flutter or opencv_dart
Local structured storage	sqlite or hive
File storage	path_provider
Pack download / unzip / verify	dio, archive, crypto
AR scan mode	ar_flutter_plugin (ARCore/ARKit underneath)
Navigation	go_router
Future hardware (BLE)	flutter_blue_plus
Background tasks	workmanager
EKF / linear algebra (later phase)	ml_linalg

Suggested build order

- Project setup, permissions, SensorHub, basic map.
- GPS, IMU, magnetometer streaming, sensor health panel.
- TrustFusionEngine v1 — jump detection, weighted confidence score.
- Map screen with colored markers and uncertainty circles.
- RegionPackManager v1 — folder structure, manifest, one sample pack.
- Camera-based landmark scan mode and manual landmark saving.
- Landmark matching v1, feeding into the fusion engine.
- Separate AR scan mode with plane/pose/quality status.
- Magnetic fingerprint recording and route comparison.
- Sextant/sky validator v1, field test bundle.